



PORTAL DE CLIENTE · DOCUMENTACIÓN TÉCNICA

EL PORTAL POST-PURCHASE DE LIT.

Cómo está construido, qué hace cada superficie, y las decisiones técnicas detrás del producto.

ÍNDICE DEL DOCUMENTO

Seis secciones que cubren producto, arquitectura, superficies, integraciones y decisiones de diseño.

PORTAL LIT
DOCUMENTACIÓN · V1.0

01

RESUMEN EJECUTIVO

Qué es el portal, para quién es, y qué problema resuelve.

P. 03

02

STACK Y ARQUITECTURA

Next.js + Vercel + Supabase + Seal. Diagrama y razones detrás de cada pieza.

P. 04

03

AUTENTICACIÓN Y SESIONES

Dos vías de auth conviviendo: App Proxy y Customer Account API OAuth.

P. 05

04

SUPERFICIES DEL PORTAL

Mi LIT, Cuenta, Skip, Plan, Cancel, Login, Address, Payment.

P. 06

05

INTEGRACIONES

Seal, Shopify Admin, Klaviyo, Supabase, App Proxy.

P. 09

06

DECISIONES Y APRENDIZAJES

Los gotchas que aprendimos por las malas y por qué la solución es la que es.

P. 11

07

MVP Y DIMENSIONES

Qué entra en V1, qué se queda fuera, números del proyecto.

P. 13

08

PRÓXIMAS FASES

Collection, Drops, Inner Circle, The World. Follow-ups técnicos.

P. 14

01 • RESUMEN EJECUTIVO

Qué es el portal, qué consigue el cliente, qué consigue LIT.

PORTAL LIT

SECCIÓN 01

El **Portal de Cliente de LIT** es la nueva área privada donde el suscriptor gestiona su caja: cambia el plan, salta envíos, modifica la dirección, añade extras, y, llegado el caso, cancela. Sustituye al portal genérico que venía con la herramienta de suscripciones (Seal), que vivía en otro dominio y rompía la identidad de marca.

EL PORQUÉ

El portal antiguo era una página técnica, fea, en otro dominio (`tracking.litsalt.com`). Tener uno propio significa que el cliente **jamás sale de litsalt.com**, ve la identidad LIT en todo momento, y nosotros controlamos cada flujo (cancelación, win-back, recompensas) en lugar de depender de lo que Seal ofrezca por defecto.

LO QUE CONSIGUE EL CLIENTE

SELF-SERVICE

Total

Saltar caja, cambiar plan, dirección y pago en un click.

SIN SALIR DE MARCA

litsalt

Todo el flujo dentro de `/apps/portal/*`.

CANCELACIÓN

4 pasos

Con alternativas antes del fin: skip, plan, sabor.

VISIBILIDAD

100%

Próximas entregas, historial, sabor, dirección.

LO QUE CONSIGUE LIT

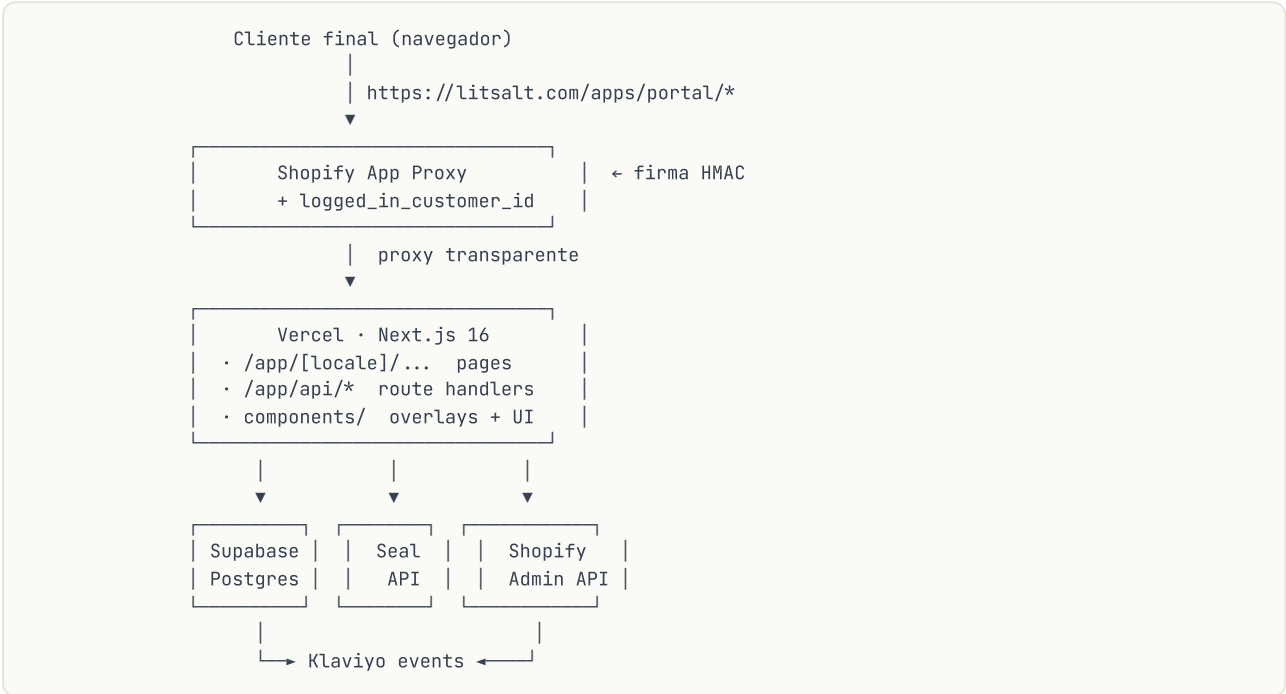
- **Reducción de tickets a soporte:** la mayoría de "skip mi caja" / "cambiar plan" pasan a self-service.
- **Marca consistente** de extremo a extremo. Misma tipografía, mismo idioma, mismo trato.
- **Recuperación de cancelaciones integrada:** el cliente ve alternativas antes de la decisión final.
- **Telemetría limpia:** cada acción se loguea en Klaviyo para alimentar flujos automáticos de retención.
- **Control sobre el roadmap:** Drops, Collection, Inner Circle, eventos. Fase 2 ya con chasis montado.

ESTADO ACTUAL

El portal está **desplegado en producción** en `litsalt.com/apps/portal/*`. Los flujos críticos (login OAuth, ver suscripción, saltar, cambiar plan, cambiar dirección, cancelar) funcionan end-to-end. Hay una auditoría de seguridad ejecutada el 21 de mayo con findings documentados aparte (`docs/SECURITY_AUDIT_2026-05-21.md`). Fases siguientes (Collection, Drops, Inner Circle) tienen schema listo en Supabase y UI con stubs.

Next.js 16 (App Router) en Vercel, servido al cliente vía **Shopify App Proxy**, con **Supabase Postgres** como base de datos, e integrado contra **Seal Subscriptions**, **Shopify Admin API** y **Klaviyo**.

— DIAGRAMA DE ALTO NIVEL



— LAS PIEZAS, UNA POR UNA

NEXT.JS 16

App Router. Server components para precargar datos sin exponer tokens al bundle. SSR para first paint rápido.

SHOPIFY APP PROXY

El cliente nunca sale de litsalt.com. Shopify firma cada request con HMAC y adjunta el customer_id.

VERCEL

Deploy continuo desde GitHub, escala automática, edge functions. Sin operar infra propia.

SUPABASE

Postgres gestionado + Auth + panel para ops review. Una sola DB para todo el portal.

SEAL SUBSCRIPTIONS

Motor de suscripciones existente. El portal habla con su Merchant API para leer y mutar.

CUSTOMER ACCOUNT API

OAuth moderno con PKCE de Shopify. Sustituye el login viejo. Soporta deep-links desde email.

DECISIÓN DE MARCA

Toda la UI usa las tipografías brand (**Barlow** y **Barlow Condensed**) y los colores LIT (**cream** #F8F9F2, **lit-grey** #323743, **bold-yellow** #EBEE62). Sin hojas de estilo neutras. Todo brand consistente.

03 · AUTENTICACIÓN Y SESIONES

Dos mecanismos de auth coexistiendo. Cualquiera identifica al cliente.

El middleware `withCustomer` intenta dos vías de identificar al cliente. La primera funciona cuando viene desde un email (App Proxy). La segunda cuando entra directamente (OAuth).

VÍA A · PASIVA

App Proxy

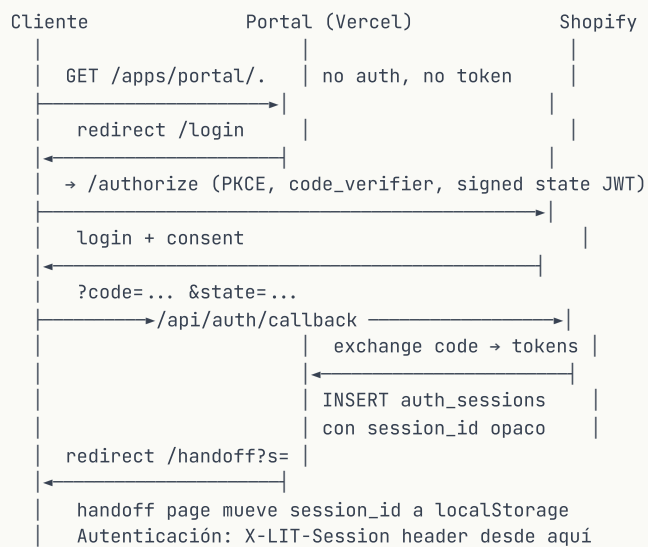
Shopify firma la URL con HMAC y adjunta `logged_in_customer_id`. Si está logueado en storefront, ya está dentro.

VÍA B · ACTIVA

OAuth (PKCE)

Customer Account API. Tras el callback creamos un `session_id` opaco en Supabase. FE lo manda en X-LIT-Session.

FLUJO OAUTH (VÍA B)



GOTCHA SHOPIFY INTERCEPTA AUTHORIZATION EN MUTATIONS

Shopify App Proxy **intercepta el header `Authorization`** en POST/PATCH/DELETE y NO las reenvía al upstream. Devuelve un 500 con el HTML del tema. Headers custom `X-*` pasan limpios. Migramos a `X-LIT-Session` y todas las mutations volvieron a funcionar.

04 · SUPERFICIES DEL PORTAL

Mi LIT (Hub) y Cuenta — las dos pantallas raíz.

PORTAL LIT

SECCIÓN 04 · 1/3

Cada superficie tiene una URL clara y una responsabilidad. Todas comparten header móvil fijo, tipografías brand y colores LIT.

4.1 · MI LIT (HUB)

HOME

RUTA PRIMARIA

/apps/portal/[es|en]/mi-lit

Pantalla principal. Estructura:

- **Hero "Next Box"**: fecha del próximo envío, sabor, plan. 4 variantes (default, skipped con banner+undo, locked dentro 24h, new primera caja).
- **Quick Actions**: 4 botones para Cambiar plan, Saltar próxima, Cambiar sabor (F2), Extras (F2).
- **Calendar**: próximas 5-6 fechas que Seal tiene pre-agendadas.
- **Mini Collection grid**: preview de la colección de cartas (Fase 2).
- **Order history**: últimos 10 pedidos completados con tracking si está disponible.

4.2 · CUENTA

ACCOUNT

SETTINGS

/apps/portal/[es|en]/cuenta

Misma información que Mi LIT en formato ficha: nombre, email, teléfono, suscripción, dirección, método de pago, historial completo. Incluye **Danger Zone** al final con "Cerrar sesión" y "Cancelar suscripción".

- Los Quick Actions funcionan exactamente igual que en Mi LIT (mismas reglas de disabled, mismo banner "Saltaste la entrega anterior — Deshacer" si hay skip activo).
- Sincronización via `localStorage` (`lit:just-skipped`): saltas en Mi LIT y navegas a Cuenta, lo ves. Y viceversa.

4.3 · SKIP OVERLAY

OVERLAY

MUTATION

POST /api/subscription/skip

Bottom-sheet en móvil, modal en desktop. Muestra "¿Te vas de viaje?", fecha actual y botón "Confirmar saltar". El backend valida ownership en Supabase (fast-path), hace GET dirigido a Seal por ID, comprueba cutoff de 24h, llama `skipBillingAttempt`, dispara Klaviyo event.

4.4 · PLAN OVERLAY

OVERLAY

MUTATION

PATCH /api/subscription/plan

Selector de cajas (1 a 6) + frecuencia (15d hasta 6 meses). Precio dinámico desde Shopify. Si hay skip activo, aviso amarillo. Al guardar:

- Si cambia frecuencia: `editSubscription(delivery_interval)` primero.
- Si cambia nº de cajas: `addItem` + `removeItems` (Seal no permite swap directo).
- Pausas de 500ms entre mutations (Seal regenera `billing_attempts` en background).
- Verify con AbortController 4s. Si timed out, respuesta sintética + re-poll silencioso 60s en FE.

4.5 · ADDRESS + 4.6 · PAYMENT

EMAIL-ONLY

PATCH /api/subscription/address · Shopify `customerPaymentMethodSendUpdateEmail`

Address: formulario con cutoff de 24h. PATCH llama a `seal.updateShippingAddress`.

Payment: 100% email-only (locked 2026-05-19). Botón dispara Shopify Admin API que envía enlace seguro al email. El portal muestra: "Te hemos enviado un enlace a tu correo, vuelve aquí al terminar". Sin Stripe, sin pasarela nueva.

04 · SUPERFICIES DEL PORTAL

El takeover de cancelación: el flujo más cuidado del portal.

PORTAL LIT

SECCIÓN 04 · 3/3

4.7 · CANCEL TAKEOVER

FULL-SCREEN

WIN-BACK

/api/subscription/cancel (steps 1 → 4)

No es un overlay, es un takeover oscuro a pantalla completa. 4 pasos:

STEP 01

Stats

"Has recibido N cajas y llevas M meses en LIT". Reconocer el viaje antes de proponer salirse.

STEP 02

Alternativas

Botones para Saltar, Cambiar plan, Sabores. Pivotan al overlay sin cerrar el takeover.

STEP 03

Motivo

Razón (caro / mucho / no lo uso / pausa / otro) + free text opcional.

STEP 04

Confirmar

Copy condicional según cutoff. Botón final "Cancelar suscripción".

— QUÉ PASA AL CONFIRMAR

1. Marca la cancellation row en Supabase como `committing` (estado intermedio para detectar cancels colgados).
2. Llama `seal.cancelSubscription` (es **inmediato**, Seal pone `cancelled_on` al momento).
3. Promueve la row a `confirmed`. Si es second-cancel: reset inmediato del balance Drops. First-cancel: hold 90 días.
4. Dispara evento Klaviyo con razón, freeText, cancelCount, fecha último envío, drops hold.
5. Muestra done state: *"Muchas gracias por haber confiado en LIT. Ojalá poder tenerte de vuelta pronto."*

SALIDA LIMPIA

El botón "Volver a LIT" del done state hace **logout completo** y redirige a `litsalt.com`. El cliente NO vuelve al portal con estado stale. Si más tarde quiere volver, el Hub le mostrará el ReactivateCard.

4.8 · LOGIN / EMPTYSTATE

FALLBACK

/apps/portal/[es|en]/mi-lit (sin auth)

Sin sesión → **LoginScreen** con CTA a OAuth. Autenticado sin sub activa (one-shot o cancelado) → **EmptyState** con "Gracias por probar LIT" + CTA a suscripción con descuento "desde el 25%".

5.1 · SEAL SUBSCRIPTIONS (MERCHANT API)

El motor de suscripciones de LIT. El portal lee y muta el estado únicamente a través de su Merchant API.

ACCIÓN	ENDPOINT	NOTAS
Listar subs por email	GET /subscriptions?page=N	Sin filtro server-side. Hay que paginar hasta 33 páginas en cuentas grandes.
Sub por ID	GET /subscription?id=X	Endpoint SINGULAR que sí honra el filtro. Evita paginar.
Cambiar intervalo	PUT /subscription action=edit	Solo delivery_interval. NO mandar billing_interval.
Cambiar variant	add_items + remove_items	No hay endpoint de swap. Añadir nuevo + quitar viejo.
Saltar attempt	PUT action=skip	Puede devolver 200 con success: false. Hay que chequear el body.
Cancelar / reactivar	PUT action=cancel reactivate	Cancel es INMEDIATO. cancelled_on al momento.

5.2 · SHOPIFY ADMIN API (GRAPHQL)

Datos del cliente y operaciones que Seal no expone:

- **Lookup de email** por customer_id — App Proxy da el ID pero Seal indexa por email.
- **Actualizar customer**: nombre, teléfono, language preference.
- **Catalog de extras**: productos con tag `add-to-box` añadibles a la siguiente caja.
- **Payment update email**: `customerPaymentMethodSendUpdateEmail`.
- **Order history**: últimos pedidos para el historial.

5.3 · KLAVIYO (TELEMETRÍA + EMAILS)

Cada acción importante dispara un evento en Klaviyo, que alimenta flujos automáticos.

EVENTO	CUÁNDO	PROPERTIES PRINCIPALES
subscription_skip	Tras skip exitoso	newNextShipDate, sealSubscriptionId
subscription_cancelled	Tras cancel step 4	primaryReason, freeText, cancelCount, lastShipDate, dropsHeldUntil
welcome_to_portal	Primera sesión OAuth	customerId, email, locale

5.4 · SUPABASE (POSTGRES)

TABLA	PROPÓSITO
auth_sessions	session_id ↔ customer_id, expiry, id_token para logout OIDC.
subscriptions	Cache customer_id ↔ seal_subscription_id. Evita el scan de 33 páginas.
cancellations	Multi-step cancel: pending → committing → confirmed.
customer_preferences	cancel_count, language_pref. Lógica "second-cancel = drops reset inmediato".
drops_balances / events	Balance Drops + log de cambios (Fase 2 activa cuando los triggers estén listos).
webhook_log	Idempotency log de webhooks Shopify y Seal.

5.5 · SHOPIFY APP PROXY

```
[app_proxy]
url      = "https://lit-portal-drab.vercel.app"
subpath  = "portal"
prefix   = "apps"
```

Cualquier request a `litsalt.com/apps/portal/*` se reenvía con HMAC + customer_id + shop + timestamp. El middleware `verifyAppProxyRequest` valida la firma antes de procesar.

GOTCHA 01 SHOPIFY APP PROXY SUSTITUYE 5XX CON HTML DEL STOREFRONT

Cuando el upstream devuelve un HTTP 5xx, App Proxy **NO reenvía esa respuesta**. Renderiza el HTML del tema con la URL como canonical. El cliente recibe ~50KB de markup con status 500 y `content-type: text/html`. **Diagnóstico:** bypass del proxy con curl directo a Vercel para ver el error JSON real.

GOTCHA 02 SEAL REGENERA BILLING_ATTEMPTS EN CADA PLAN CHANGE

Cualquier `editSubscription` en Seal regenera la lista de `billing_attempts` desde cero. **Cualquier skip aplicado previamente se borra silenciosamente.** Mitigación: el FE limpia el flag `justSkipped` de localStorage cuando hay plan change exitoso, y el PlanOverlay avisa al usuario si va a cambiar plan con un skip activo.

GOTCHA 03 EVENTUAL CONSISTENCY EN SEAL

Tras una mutation, un GET inmediato puede devolver datos stale. Soluciones: (1) el skip endpoint **computa el nuevo nextShipDate localmente**; (2) el plan endpoint usa AbortController con timeout de 4s; (3) el FE arranca re-poll silencioso de 60s tras un plan change.

GOTCHA 04 CONSTRAINT CHECK SIN MIGRATION APLICADA EN PROD

El código de cancel step 4 escribía `status='committing'` pero la constraint solo permitía `('pending', 'confirmed')`. Durante semanas **nadie podía cancelar**. El 500 se enmascaró por el Gotcha 01. Fix: ALTER CONSTRAINT aplicado 2026-05-21. Schema sincronizado.

GOTCHA 05 CANCEL ES INMEDIATO EN SEAL

El comportamiento por defecto de Seal cancel pone `cancelled_on` al instante, no tras el último envío. El portal refleja esto en el copy según el cutoff: *"Cancelación inmediata"*, excepto si el próximo envío ya está procesando dentro de 24h.

REGLA 06 EL CLIENTE JAMÁS VE TRACKING.LITSALT.COM

El cliente final no puede ver el portal de tracking antiguo en ningún momento. Todo dentro de `litsalt.com/apps/portal/*`. Cambio de pago = email. Cancel = takeover. Error = inline. El portal viejo deja de existir para el cliente.

REGLA 07 SIN EM-DASHES EN NINGÚN COPY

Ningún copy de LIT (portal + emails + ads) puede llevar un em-dash. Usar coma o punto. Preferencia consistente del founder.

REGLA 08 CUTOFF DE 24H (ANTES 72H)

El operador interno necesita al menos 24h para procesar cambios antes del envío. Antes eran 72h, que bloqueaba demasiados cambios. Se aplica a skip, plan, address, extras. Constante única en `src/lib/cutoff.ts`.

REGLA 09 CAMBIO DE MÉTODO DE PAGO = EMAIL-ONLY

100% email-only via Shopify `customerPaymentMethodSendUpdateEmail`. Sin Stripe, sin pasarela nueva. Shopify ya tiene el método y la pasarela.

REGLA 10 X-LIT-SESSION, NO AUTHORIZATION

El portal envía el session token en header custom `X-LIT-Session`. El backend acepta ambos para back-compat, pero el FE solo usa el custom. Esto resolvió de raíz "la mutation parece que funciona pero nunca llega al servidor".

CIERRE

El portal está vivo, los flujos críticos funcionan end-to-end, y el cliente puede gestionar su suscripción sin ver el portal antiguo. La base es sólida. Las decisiones documentadas en estas páginas son las que evitarán que el equipo repita los mismos errores en Fase 2 (Collection, Drops, Inner Circle) o cuando se monte un portal para otra marca con la misma arquitectura.

QUÉ ENTRA EN EL MVP

El MVP del portal cubre los flujos imprescindibles para que un suscriptor gestione su caja sin tocar soporte. Todo lo demás (Collection, Drops, eventos) queda como Fase 2 con el chasis ya montado.

- **Login OAuth** con Customer Account API de Shopify (PKCE + nonce + JWKS verification).
- **Mi LIT (Hub)**: hero con próximo envío, quick actions, calendar de próximas entregas, historial de pedidos.
- **Cuenta**: ficha de cliente, suscripción, dirección, método de pago, cancelación.
- **Skip / Undo**: saltar próxima caja, deshacer en ventana de 5 minutos.
- **Plan change**: cambiar cajas y/o frecuencia con precio dinámico desde Shopify.
- **Cancel**: takeover en 4 pasos con alternativas (skip, plan, sabor) antes del fin.
- **Reactivate**: ventana de 90 días tras primera cancelación.
- **Address change**: cambio de dirección de envío vía Seal directo.
- **Payment update**: 100% email-only via Shopify `customerPaymentMethodSendUpdateEmail`.
- **Email change**: con verificación por magic link al nuevo email.
- **EmptyState**: para clientes sin suscripción activa (one-shot buyers, cancelados).
- **Bilingüe ES/EN**: idioma persistido + URL `/es/*` o `/en/*`.
- **Telemetría Klaviyo**: eventos para `subscription_skip`, `subscription_cancelled`, `welcome_to_portal`, `email_change_requested`, `subscription_reactivated`.
- **Seguridad cerrada**: 13 hallazgos críticos/altos auditados y resueltos. Re-audit confirma cero regresiones.

DIMENSIONES

LÍNEAS DE CÓDIGO 14k TypeScript + TSX en src/	ENDPOINTS API 41 routes en src/app/api/	COMPONENTES REACT 25 UI atómicos en src/components/	MIGRATIONS EN PROD 04 cancellations, auth_sessions_hash, email_change_requests, rate_buckets
---	---	---	---

TABLA DE SUPERFICIES Y ENDPOINTS

CAPA	CUENTA	NOTAS
Superficies activas	08	Hub, Cuenta, Skip, Plan, Cancel, Address, Login/Empty, Confirmación post-cancel
Superficies Fase 2 (UI con stub)	03	Collection, The World, Inner Circle
Tablas Supabase	13+	auth_sessions, subscriptions, cancellations, customer_preferences, drops_*, events, moments, stories, webhook_log, rate_buckets, email_change_requests
Integraciones externas	05	Seal, Shopify Admin, Klaviyo, Customer Account API OAuth, App Proxy
Eventos Klaviyo	10	skip, cancelled, reactivated, email_change_requested, welcome_to_portal, tier_unlocked, reward_claimed, winback_d14/d30, drops_earned

FASE 2 – ENGAGEMENT & LOYALTY

BLOQUE	ESTADO	QUÉ FALTA
Collection (cartas)	UI stub en Hub	Backend de unlocks (drops_events trigger por box shipped), grid completa en /collection, animación de carta nueva.
Drops (recompensas)	Schema listo	Triggers: box_shipped (+10), referral_converted (+50), monthly_streak (+5), product_review (+15). Rewards claim flow + carrito de recompensas.
Inner Circle (tier)	Pill visible	Threshold de unlock (X cajas o Y meses), beneficios visibles, evento Klaviyo tier_unlocked ya está pero no se dispara nada.
The World (eventos)	Schema + 1 endpoint	UI completa /the-world, calendario, RSVP, lista de espera Barcelona, integración con admin para crear eventos.
Flavor switcher	QA disabled	Activar cuando haya catálogo >1 sabor. La logic de mutation ya existe (es un plan change variant).
Extras catalog	Backend completo	UI activar cuando Shopify tenga productos con tag <code>add-to-box</code> .

HARDENING PENDIENTE (LOW PRIORITY)

- Drop columna `session_id` raw de `auth_sessions`. Hoy se conserva por rollback safety; tras unas semanas estable se puede tirar.
- Drop `auth_sessions.email` si no se usa (defense in depth — DB leak no debería exponer emails).
- Rate limit en `/api/customer/confirm-email`: token de 256 bits hace que la probabilidad de adivinar sea ~0, pero limitar IP cubre el caso de scanning.
- Comentar `trust model` del logout `id_token` decode (cosmético — el auditor lo notó).
- Auditoría bimensual automática: cron programado para 2026-06-05; cuando dispare, los 3 agentes ejecutan y se auto-reagenda.

MEJORAS DE OPERACIONES

- Dashboard ops para ver cancellations en estado `committing` (cancels colgados que requieren intervención manual).
- Sentry o equivalente para tracking de errores en producción. Hoy todo va a Vercel logs sin alerting.
- E2E tests con Playwright para los flujos críticos: skip, plan change, cancel, address.
- Webhook log replay tool para reprocesar eventos Shopify/Seal fallidos.
- Linear integration: pipe del audit bimensual a tickets automáticos si hay críticos.

ESTADO A FECHA DE HOY

El portal está **en producción y listo para lanzamiento al cliente final**. Los flujos críticos están validados end-to-end, la auditoría de seguridad ha quedado limpia, y la cadencia de re-audit bimensual está programada. La Fase 2 (Collection / Drops / Inner Circle) tiene schema y stubs en su sitio — no es un sprint nuevo de arquitectura, es un sprint de relleno.



FIN DEL DOCUMENTO

GRACIAS POR LEER.

PORTAL LIT · DOC V1.0 · 21 · 05 · 2026